

Part 1: Functional Requirements (FR)

FR1: Timetable Data Ingestion & Integrity Check

- User Story: As an SCIT operations officer, I want to upload semester timetabling datasets exported from UOW Enterprise so that the system can automatically establish a verified operational baseline.
- Acceptance Criteria:
 - System accepts .xlsx and .csv files containing standard Enterprise export fields (subject codes, room numbers, class durations, delivery weeks, cohort sizes).
 - Script flags missing values, unrecognized room codes and malformed time blocks before data insertion.
 - System outputs a simple summary log indicating successful record count and any invalid rows.

FR2: Lab Utilisation & Spatial Analytics

- User Story: As school management, I want to see the real utilization rates across computer labs so that we know which rooms are over-booked or sitting empty.
- Acceptance Criteria:
 - Computes weekly room utilization based on the agreed formula:

$$\text{Lab Utilisation (\%)} = \frac{\text{Booked Lab Hours}}{\text{Available Lab Hours}} \times 100$$

reported per laboratory and per teaching week.

- Generates an interactive table showing available hours, booked hours and utilization percentages, with an option to export results to Excel.
- Displays a color-graded Heatmap matrix (Labs vs Teaching Weeks/Time Slots) to quickly spot bottlenecks.

FR3: Multi-Variable Scenario Simulation Engine

- User Story: As a subject coordinator, I want to model proposed curriculum changes without altering current schedules so that I can see their feasibility beforehand.
- Acceptance Criteria:
 - Duplicates baseline schedule data into an isolated scenario sandbox without modifying original records.
 - Allows adjustments to key variables: class lengths (1h vs 2h), session frequency (weekly vs fortnightly), class sizes and room capacities (such as 25-seat vs 48-seat layouts).

- Applies constraint logic: strictly enforces zero double-bookings (Hard Constraint) while testing room fit and schedule compacting (Soft Constraints).

FR4: Side-by-Side Comparison & Recommendation Panel

- User Story: As SCIT leadership, I want to compare simulated scenarios against our baseline with clear operational justifications to support curriculum planning decisions.
- Acceptance Criteria:
 - Displays side-by-side delta metrics: net changes in booked lab hours, room clash counts and teaching hours per subject.
 - Features a clear Recommendation Panel highlighting trade-offs, capacity shortfalls and the reasoning behind each suggestion.
 - Where cohort sequence data allows, tracks potential reductions in student on-campus days.

Part 2: Design & Technical Requirements

1. Design Requirements (UX & Non-Functional Guidelines)

- Target Audience Fit (Decision-Support Focus):
 - The tool is built strictly for School Operations and Academic Subject Coordinators, not as a general booking portal.
 - Interface prioritizes high-level summary tables, clear heatmaps and scenario comparison cards, avoiding unnecessary administrative booking overhead.
- Computational Accuracy:
 - 100% precision on total contact hours and room double-booking checks. Infeasible scenario runs must return explicit clash warnings rather than broken schedules.
- Data Security & Privacy:
 - Implements Role-Based Access Control (RBAC). Room data and internal timetabling spreadsheets remain locally managed and restricted to authorized school staff, complying with university data requirements.
- Ethical Compliance (ACS Code of Ethics):
 - The tool assists human decision-makers rather than replacing their judgment. Recommendations are fully transparent with visible trade-offs so academic staff retain final oversight.

2. Technical Requirements & Stack Justification

- Front-End (React + TypeScript + Plotly.js):

- Why chosen: React provides modular UI state management; TypeScript eliminates runtime typing bugs in complex timetable models; Plotly.js handles fast client-side rendering for multi-week heatmaps and comparison charts.
- Back-End (Python + FastAPI):
 - Why chosen: FastAPI offers high execution speed, native asynchronous handling, automatic Swagger/OpenAPI documentation and simple integration with data manipulation libraries.
- Simulation Engine (Google OR-Tools CP-SAT):
 - Why chosen: Timetable scheduling is an NP-hard problem. OR-Tools CP-SAT efficiently handles constraint satisfaction, room capacity bounds and infeasibility detection without needing a proprietary solver.
- Data Layer (pandas + PostgreSQL / SQLite):
 - Why chosen: pandas and openpyxl parse and clean messy spreadsheet exports from Enterprise; PostgreSQL stores finalized baseline datasets and saved scenarios; SQLite handles local development and integration tests.
- Deployment & Collaboration (Docker + Git):
 - Why chosen: Containerization guarantees environment consistency across all developer machines; Git feature branches prevent unreviewed merges.

Part 3: Initial UI Designs & User Interaction Flow

1. User Interaction Flow (5-Step Operational Pipeline)

1. Step 1: Ingest & Validate:

User logs in → Uploads semester .xlsx/.csv export → System runs validation rules and displays clean ingestion status.

2. Step 2: Inspect Baseline:

Dashboard displays current semester metrics: overall lab utilisation %, bottlenecked labs and peak time slots via the baseline heatmap.

3. Step 3: Configure Scenario:

User clicks "New Scenario" (cloning baseline data) → Adjusts parameters via sliders and dropdowns (such as convert CSIT labs from 1h weekly to 2h fortnightly, test 48-seat vs 25-seat capacities) → Clicks "Run Simulation".

4. Step 4: Evaluate Comparison:

System displays a split-screen view: Baseline vs New Scenario, showing changes in total hours, room utilization % and subject teaching hours.

5. Step 5: Review Insights & Export:

User reads specific constraint warnings or layout suggestions in the Recommendation Panel
→ Exports summary tables directly to Excel for committee discussions.

2. Core Screen Layouts